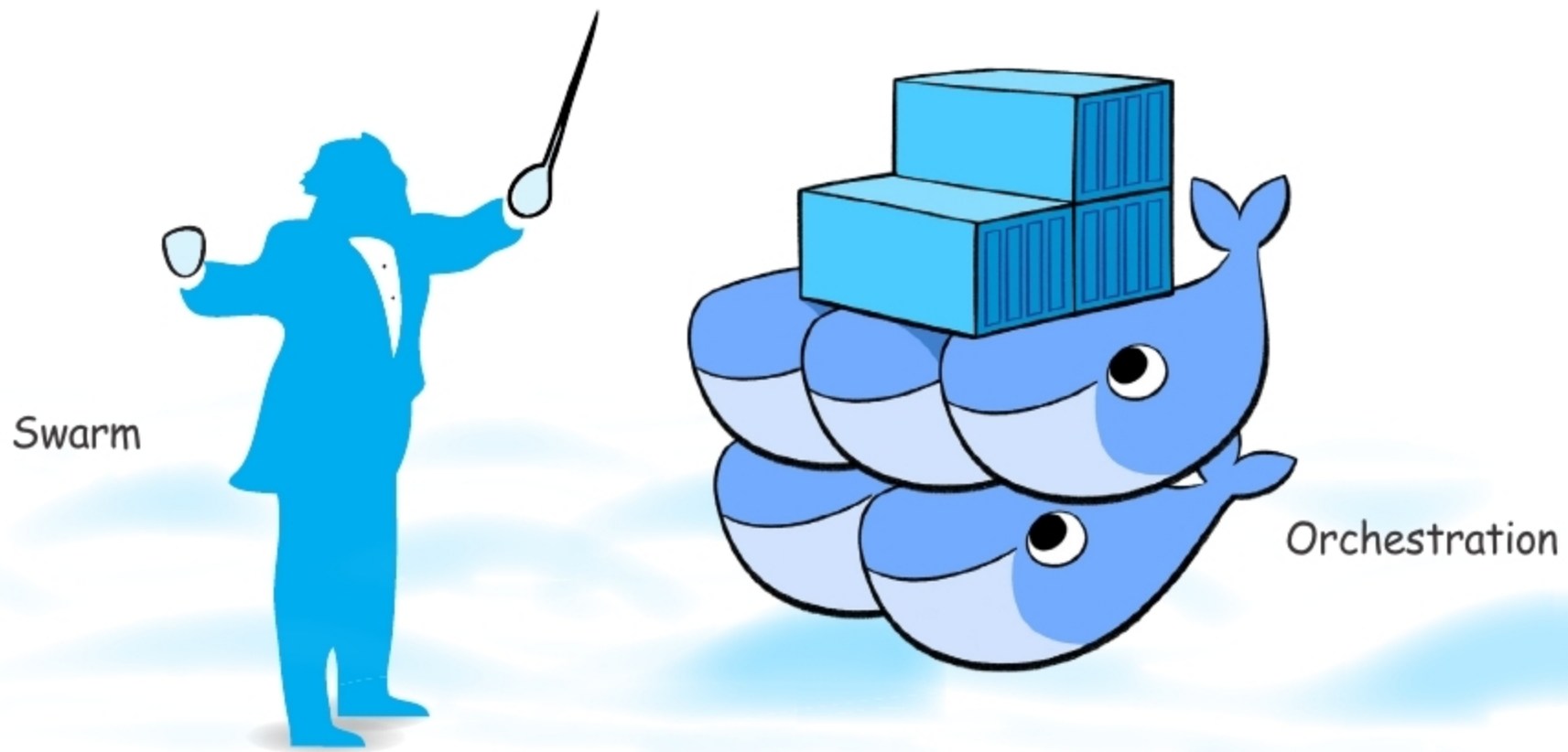


Dynamic Test Orchestration Using Docker Swarm



This article covers various aspects of test orchestration in Docker Swarm. It explains the running of applications in Docker Swarm, and guides the reader through the process of testing while scaling containers up or tearing them down.

Monolithic applications are moving into microservices based technologies at a faster rate than ever. Many such bigger environments are using Docker Swarm or Kubernetes technologies for container orchestration and monitoring. These large environments always need testing to be carried out before going into production. Testing in such dynamic and fast growing environments is challenging, and needs extra effort and caution. This article will address such configuration challenges, scalability aspects and the handling of Docker Swarm as the environment.

Docker Swarm and the need for orchestration

As explained earlier, the most important aspect of Docker containers is to achieve scalability. Achieving scalability is a huge challenge in non-Docker environments, which is why people prefer Docker containers. When maintaining just a few containers and fewer services/applications, there are no issues, but running thousands of container images and hundreds of services would make such scenarios complicated. That is when we see the potential need for orchestration in containers.

Orchestration is needed to automate the deployment, management, scaling, networking and availability check of the containers. Again, as explained in the introductory

section, especially in large and dynamic test environments, test teams or the entire operations teams can use container orchestration to control the following tasks:

- Provisioning of containers
- Deployment of containers
- Availability of containers
- Allocation of resources between containers
- Movement of containers from one host to another in case of any host/hardware failures
- Scaling up (or down) containers for equivalent application spreads
- Load balancing of services across the containers
- Movement of test data and test cases while the containers are running
- Bulk transfer of test data or configuration data for all containers

As mentioned, we have selected Docker Swarm to do the job of orchestration. This orchestrator is bundled with Docker containers, a feature inbuilt into this stream, and this technology is extremely simple to use and configure. For small tests, development teams and deployments, Docker Swarm is the preferred choice. For large tests and development teams and deployments, we can opt for